

CS 452/552 Operations Systems

Project 2 The Classic Bounded Buffer Problem

Name: Tanmai Kumar Ghosh

Bronco ID: 114224102

Date: 10/22/2024

1. Project Overview

In this project, we will build a FIFO queue using a monitor to address the bounded buffer problem. The queue will have a set capacity, blocking threads when it is full or empty. Bounded buffers play a crucial role in Operating Systems. Although programming in languages like Java, Python, or C# may give the illusion of unlimited memory, it's merely an abstraction provided by the OS—real memory is limited by hardware. This queue can serve as the foundation for creating more advanced features, such as a thread pool.

2. Project Management Plan

The project starts with the initial code base with necessary configuration.

(a) Task 1: Initialize Code Starter

- (i) Clone the Starter code from the Repository. This time we couldn't fork it as we forked the project before with the same name.

git clone git@github.com:JianshuLiu1721/cs452-project-starter.git

- (ii) I created my own repository and pushed the starter codes in the repository.

The github repo: <https://github.com/tanmaibsu/OS-Buffer-Problem>

- (iii) Now, the codebase is ready for the further development and can initiate the codespace.

(b) Task 2: Queue structure (Data Structure)

```
struct queue
{
    void **data;
    int capacity;
    int front;
    int rear;
    int size;
    bool shutdown;
    pthread_mutex_t lock;
    pthread_cond_t cond;
};
```

- (i) **void **data:** An array of pointers used to store the elements of the queue.
- (ii) **int capacity:** The maximum number of elements the queue can hold.
- (iii) **int front:** The index of the front (first) element in the queue.
- (iv) **int rear:** The index of the rear (last) element in the queue.
- (v) **int size:** The current number of elements in the queue.
- (vi) **bool shutdown:** A flag indicating whether the queue is in shutdown mode.

(c) Task 3: Initialize Queue

(i) **queue_init(int capacity)**

1. Allocates memory for the queue structure using **malloc(sizeof(struct queue))**. If allocation fails, it returns NULL.
2. Allocates memory for the queue's data array (**q->data**) to hold up to capacity elements. If this allocation fails, it frees the queue structure and returns NULL.
3. Sets the queue's capacity (**q->capacity = capacity**) to the provided value.
4. Initializes the front and rear indices: **q->front = 0** to indicate the starting position for dequeue operations. **q->rear = -1** since no elements have been enqueued yet.
5. Sets the initial size of the queue to 0 (**q->size = 0**).
6. Initializes the shutdown flag (**q->shutdown = false**), meaning the queue is ready for operations.
7. Initializes the mutex lock (**pthread_mutex_init(&q->lock, NULL)**) to protect the queue during concurrent access.
8. Initializes the condition variable (**pthread_cond_init(&q->cond, NULL)**) for signaling between threads.

9. Returns the initialized queue if all steps succeed.

(d) Task 4: Enqueue() and Dequeue() Implementation

(i) enqueue(queue_t q, void *data)

1. Locks the queue using **lock_queue(q)** to ensure thread-safe access.
2. Waits if the queue is full (**q->size == q->capacity**) and the queue has not been shut down (**!q->shutdown**).
3. Exits if the queue is in shutdown mode (**q->shutdown**).
4. Inserts the new element at the rear of the queue, updating the rear index using modulo arithmetic to handle wrapping.
5. Increments the size of the queue (**q->size++**).
6. Signals waiting threads that space is now available in the queue (**signal_condition(q)**).
7. Unlocks the queue after completing the operation.

(ii) dequeue(queue_t q)

1. Locks the queue using **lock_queue(q)** for thread-safe access.
2. Waits if the queue is empty (**q->size == 0**) and the queue has not been shut down (**!q->shutdown**).
3. Exits if the queue is in shutdown mode and empty (**q->shutdown && q->size == 0**).
4. Removes the front element from the queue, saving it to data and updating the front index using modulo arithmetic.
5. Decreases the size of the queue (**q->size--**).
6. Signals waiting threads that an element has been removed (**signal_condition(q)**).
7. Unlocks the queue and returns the dequeued element.

3. Project Deliveries

a. Code Compilation

- i. Open Terminal
- ii. Clean the build with **make clean**
- iii. Build the implementation with **make**
- iv. If you change anything in the implementation, first **make clean** and run **make** again.
- v. Run **./myprogram** for the final output.
- vi. Run **./test-lab** for the test results.

b. Self-modification

- i. I did not make any modification except in the **src/lab.c** implementing the functions defined in **src/lab.h**.

c. Summary of Results

i. Results of the program

```
● @tanmaibsu → /workspaces/OS-Buffer-Problem (main) $ ./myprogram
Simulating 1 producers 1 consumers with 10 items per thread and a queue size of 5
Creating 1 consumer threads
Queue is empty:true
Total produced:10
Total consumed:10
1.122070 10
```

ii. Test Results

```
● @tanmaibsu → /workspaces/OS-Buffer-Problem (main) $ ./test-lab
tests/test-lab.c:78:test_create_destroy:PASS
tests/test-lab.c:79:test_queue_dequeue:PASS
tests/test-lab.c:80:test_queue_dequeue_multiple:PASS
tests/test-lab.c:81:test_queue_dequeue_shutdown:PASS

-----
4 Tests 0 Failures 0 Ignored
OK
```

4. Self-Reflection

I have gained practical insights into key concepts of Operating Systems, specifically about buffer queues and thread synchronization.

- (a) Bounded Buffer (Queue Implementation):** The code demonstrates how to implement a bounded buffer using a queue structure, which has a fixed capacity. This is crucial in systems that need to handle limited resources like memory. The queue's behavior is controlled based on its capacity and size, allowing for safe insertion (enqueue) and removal (dequeue) of elements, even when the queue is full or empty.

(b) Thread Synchronization with Mutexes and Condition Variables:

- i. **Mutex Locks:** The code uses `pthread_mutex_t` to ensure mutual exclusion, meaning only one thread can access the shared queue at a time, preventing race conditions.
- ii. **Condition Variables:** `pthread_cond_t` is used to allow threads to wait and signal changes in the queue state (e.g., when space becomes available or when elements are added). This ensures that threads waiting on an empty or full queue can proceed when conditions change.

(c) Blocking and Wake-up Mechanisms: The code introduces how threads are blocked when the queue is full during an enqueue operation or empty during a dequeue operation. This teaches the concept of efficient resource management, where threads wait instead of continuously checking the queue's state. By signaling waiting threads with `pthread_cond_signal` or `pthread_cond_broadcast`, the program ensures that threads can resume their operations when the conditions change (e.g., an element is added or removed).

(d) Shutdown Behavior: I have also learned about safely shutting down a queue, where operations can be gracefully stopped by setting a shutdown flag and waking up all waiting threads with `pthread_cond_broadcast`. This is useful for controlled termination in multithreaded environments.

(e) Challenges I faced: Everytime I made a change in the code needs to clean the build files and build the codes again. I also faced challenges to resume the threads.

(f) Known Issue: While executing the build file, it may occur address sanitizer issue though in rare cases. Please use `ctrl-c` and rerun the project. I tried a lot to solve it but did not get any luck.

Overall, I have applied understanding of how to handle concurrency and synchronization in operating systems using threads, bounded buffers, and queue data structures.